

Coverage Dashboard

POC Requirement:

Build any services in Vue and Golang with unit tests for both frontend and backend, capture coverage percentages, and send the service name, coverage data, and timestamp to a backend API that stores it in a database. Display results in a UI with date-based filtering, date range selection, CSV download, and metrics including total test cases, pass percentage, failure test count, and average execution time.

GitHub Link : <https://github.com/iamdebopriya/POC-Coverage-Service>

1. What is the Coverage Dashboard?

Objective:

Coverage Dashboard is a standalone service that runs on its own — completely separate from the other services.

The Coverage Dashboard solves this. It is one web page that:

- Shows a dropdown of all the services
- Runs the tests for whichever service you pick — with one click
- Streams the test output live in a terminal window on screen
- Saves the results (pass/fail counts, coverage %) to a database automatically
- Shows a history table of all past runs so you can track progress over time

The Big Picture:

Here is how everything connects:



The Coverage Dashboard reads the services' folders, runs their tests, and shows you the results — all without touching the service code.

2. How the Folders are Set Up

All the projects sit in the same root folder. The Coverage Dashboard is just one of those projects.

```
poc2_ami/                                     ← the root folder
├── coverage-service/                          ← THIS app
│   ├── backend/                             ← Go server (port 8081)
│   │   ├── main.go
│   │   ├── config/
│   │   │   └── services.json                ← add the services here
│   │   └── handlers/run_script.go          ← runs the tests
│   │   ├── .env                            ← database settings
│   └── frontend/                           ← Vue dashboard (port 5173)
├── hello-world/                             ← service 1
│   ├── backend/                             ← go test runs here
│   └── frontend/                           ← npm test runs here
└── inventory-tracker/                       ← service 2
    ├── backend/
    └── frontend/
```

Key point

The Coverage Dashboard only reads the services' folders to run tests. It never changes any of the service code.

3. Registering a Service

To make a service appear in the dropdown, you add it to one file:

```
coverage-service/backend/config/services.json
```

Here is what that file looks like:

```
[
  {
    "name": "hello-world",
    "display_name": "Hello World",
    "backend_path": "../../hello-world/backend",
    "frontend_path": "../../hello-world/frontend",
    "backend_type": "go",
    "frontend_type": "npm"
  },
  {
    "name": "inventory-tracker",
    "display_name": "Inventory Tracker",
    "backend_path": "../../inventory-tracker/backend",
    "frontend_path": "../../inventory-tracker/frontend",
    "backend_type": "go",
    "frontend_type": "npm"
  }
]
```

What each field means

Field	What it does
name	A short ID with no spaces. Used internally.
display_name	The label shown in the dropdown on the dashboard.
backend_path	Path to the Go project folder. Relative to the backend/ folder.
frontend_path	Path to the Vue project folder. Relative to the backend/ folder.
backend_type	"go" — the runner will use go test ./...
frontend_type	"npm" — the runner will use npx vitest run

How to add a new service — 4 steps

1	Open <code>coverage-service/backend/config/services.json</code>
2	Copy one of the existing entries and paste it below
3	Update <code>name</code> , <code>display_name</code> , <code>backend_path</code> , and <code>frontend_path</code>
4	Save the file and restart the backend — done!

That is all

No other code changes needed. The new service appears in the dropdown automatically.

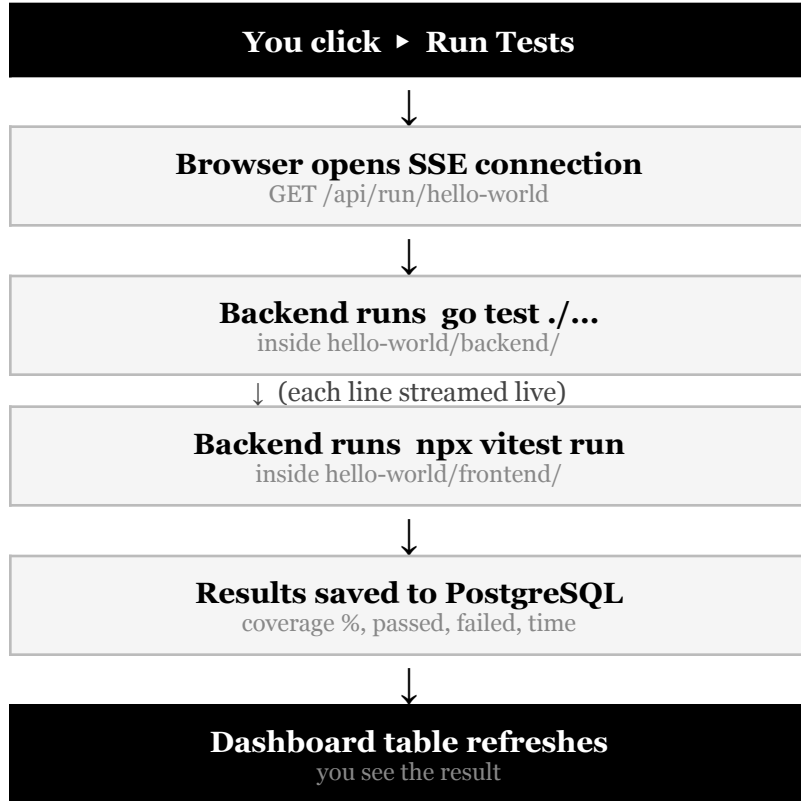
4. What Happens When You Click Run

When you select a service and click Run Tests, seven things happen automatically — you do not need to do any of them:

1	The browser opens a live connection to the backend (called SSE — Server-Sent Events).
2	The backend finds the service you picked in <code>services.json</code> and reads its folder paths.
3	The backend runs <code>go test ./... -v -cover</code> inside the Go (backend) folder of that service.
4	Each line of test output is sent to browser instantly — you see it appear in real time.
5	The backend also runs <code>npx vitest run</code> inside the Vue (frontend) folder. That output streams too.
6	When both finish, the backend counts pass/fail and extracts the coverage %. It saves everything to the database.
7	The live connection closes. The results table on page refreshes automatically.

Flow Diagram

Here is the same flow as a diagram:



5. How to Run It

You need two terminals open — one for the backend, one for the frontend.

Step 1 — Start the backend

```
cd coverage-service/backend
go run main.go
```

You should see:

```
Database connected successfully!
Coverage service running on port: 8081
```

Step 2 — Start the frontend

Open a second terminal:

```
cd coverage-service/frontend
npm install
npm run dev
```

You should see:

```
VITE v5.x ready
Local: http://localhost:5173/
```

Step 3 — Open the dashboard

```
Open browser and go to:
http://localhost:5173
```

TEST COVERAGE TRACKER

Coverage Dashboard

Run tests, track BE & FE coverage, and monitor pass/fail results across your services.

[Go to Dashboard](#) → [Select a service to get started](#)

70% +
Avg BE Coverage

70% +
Avg FE Coverage

Run Tests

Select any service and trigger a test run directly from the dashboard. Watch live output stream in real time.



BE & FE Coverage

Track backend and frontend coverage separately per service, with pass rate and average run time.

Export & Filter

Filter runs by service or date range. Export the full history as a CSV with one click.

/dashboard

● List Of Services:

← Back

TEST COVERAGE TRACKER

● Live

Coverage Dashboard

RUN TESTS

Run Tests

Inventory Tracker

▶ Run Tests

Select a service...

Hello World

Inventory Tracker

● Service - Inventory Tracker

← Back

TEST COVERAGE TRACKER

● Live

Coverage Dashboard

OVERVIEW

AVG BE COVERAGE

73.8%

AVG FE COVERAGE

76.3%

TOTAL TESTS

51

PASSED

51

all runs

FAILED

0

PASS RATE

100.0%

RUNS

1

SERVICES

2

RUN TESTS

Run Tests

Inventory Tracker

▶ Run Tests

✓ Done

EditItem.vue	69.23	83.33	12.5	68	23,35,45,57,90-96
Home.vue	98.79	93.05	95.83	98.68	137
Landing.vue	66.66	50	0	60	17-20
Login.vue	57.89	82.35	25	55.55	23,37,57-63
Register.vue	77.77	83.33	66.66	76.92	48,79-90
services	48.62	0	0	56.52	
api.js	48.62	0	0	56.52	12-16,34-37,40-43

BE Coverage: 73.8%

FE Coverage: 76.3%

Total Tests: 51

Passed: 51

Failed: 0

Time: 16.72s

✓ Results saved to dashboard

LATEST RUN

inventory-tracker BE 73.8% FE 76.3% 51/51 Passed 5/20/2026, 10:54:20 PM

FILTERS

SERVICE

All Services

FROM

dd-mm-yyyy --:--

TO

dd-mm-yyyy --:--

Apply Filter

Export CSV

COVERAGE HISTORY

SERVICE	BE COV.	FE COV.	TOTAL	PASSED	FAILED	FLAKY	PASS RATE	AVG TIME (S)	TIMESTAMP
inventory-tracker	73.8%	76.3%	51	51	0	0	100.0%	16.72	5/20/2026, 10:54:20 PM

• Service-Hello World

← Back

TEST COVERAGE TRACKER

Coverage Dashboard

● Live

OVERVIEW

AVG BE COVERAGE
80.7%

AVG FE COVERAGE
77.3%

TOTAL TESTS
66

PASSED
66
all runs

FAILED
0

PASS RATE
100.0%

RUNS
2

SERVICES
2

RUN TESTS

Run Tests

Hello World

▶ Run Tests

✓ Done

All files	78.37	80	33.33	78.37
src	0	0	0	0
App.vue	0	0	0	0 1-28
main.ts	0	0	0	0 1-4
src/components	100	92.3	100	100
HelloWorld.vue	100	92.3	100	100 26

=====

BE Coverage: 87.5%

FE Coverage: 78.4%

Total Tests: 15

Passed: 15

Failed: 0

Time: 8.65s

=====

✓ Results saved to dashboard

LATEST RUN

hello-world BE 87.5% FE 78.4% 15/15 Passed 5/20/2026, 10:55:09 PM

FILTERS

SERVICE

FROM

TO

All Services

dd-mm-yyyy --:--

dd-mm-yyyy --:--

Apply Filter

Export CSV

COVERAGE HISTORY

SERVICE	BE COV.	FE COV.	TOTAL	PASSED	FAILED	FLAKY	PASS RATE	AVG TIME (S)	TIMESTAMP
hello-world	87.5%	78.4%	15	15	0	0	100.0%	8.65	5/20/2026, 10:55:09 PM
inventory-tracker	73.8%	76.3%	51	51	0	0	100.0%	16.72	5/20/2026, 10:54:20 PM

6. Reading the Results

After a run completes, the results table shows one row per test run. Here is what each column means:

Column	What it shows
Service	Which service was tested
BE Cov.	% of the Go (backend) code covered by tests
FE Cov.	% of the Vue (frontend) code covered by tests
Total	Total number of test cases that ran
Passed	How many tests passed (green is good)
Failed	How many tests failed (red means action needed)
Pass Rate	Passed ÷ Total — a quick health score
Avg Time	How long the whole test run took in seconds
Timestamp	Exact date and time the run finished

Coverage colour guide

Colour	Coverage	What it means
● Green	85% and above	Great — tests cover most of the code
● Orange	50% to 84%	Okay — could write a few more tests
● Red	Below 50%	Low — more tests needed

Filtering and exporting

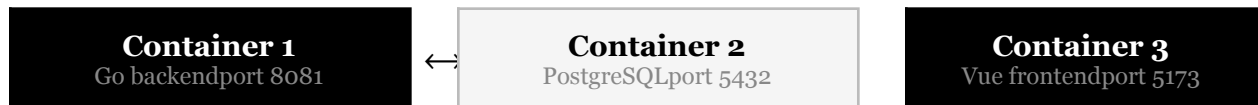
- Use the filter bar to narrow results by service name or date range
- Click the CSV button to download results as a spreadsheet

7. Docker

What Docker does

It packages the Go backend, the Vue frontend, and PostgreSQL into three containers. One command starts all three at once.

What the setup will look like



The browser talks to Container 3 (the frontend), which talks to Container 1 (the backend), which reads Container 2 (the database).

Docker files

- docker-compose.yml — starts all three containers
- backend/Dockerfile — builds the Go binary into a tiny Alpine image
- frontend/Dockerfile — builds the Vue app and serves it with nginx
- frontend/nginx.conf — routes /api/ requests to the backend; required for SSE streaming to work

The command (once Docker is set up)

```
cd coverage-service
docker compose up --build
```

Then open <http://localhost:5173> — everything is running.

```
[+] up 3/3
✓ Container coverage-service-db-1      Running
✓ Container coverage-service-backend-1 Running
✓ Container coverage-service-frontend-1 Running
Attaching to backend-1, db-1, frontend-1
Container coverage-service-db-1 Waiting
Container coverage-service-db-1 Healthy
```

One thing to remember

When running in Docker, the backend needs access to the other service folders (hello-world, inventory-tracker). This is done by mounting the poc2_ami folder as a Docker volume so the backend can find and run their tests.

8. Quick Reference

Task	Command or location
Start backend	<code>cd coverage-service/backend → go run main.go</code>
Start frontend	<code>cd coverage-service/frontend → npm run dev</code>
Open dashboard	<code>http://localhost:5173</code>
Add a new service	Edit <code>services.json</code> and restart the backend
Backend port	8081
Frontend port	5173
Database config	<code>coverage-service/backend/.env</code>
Run with Docker	<code>docker compose up --build</code>

Docker Commands

<code>docker compose up --build</code>	First time or after code changes
<code>docker compose up</code>	Start everything (no rebuild)
<code>docker compose start</code>	Resume stopped containers (fastest)
<code>docker compose stop</code>	Stop but keep data
<code>docker compose down</code>	Stop and remove containers (data safe in volume)
<code>docker compose down -v</code>	Stop and delete everything including database
<code>docker compose restart backend</code>	Restart only backend (e.g. after <code>services.json</code> change)
<code>docker compose ps</code>	Check what is running

POC Completion Summary

	Requirement	Status
1	Build a Hello World or any other service using Vue and Golang	Done
2	Write unit tests for both frontend and backend	Done
3	Keep tests in separate folder	Done
4	Execute tests and capture coverage % for both	Done
5	Create a script to run tests and collect coverage	Done
6	Send service name, backend coverage, frontend coverage, timestamp to backend API	Done
7	Store the data in DB	Done — PostgreSQL via GORM
8	Display results in a simple UI	Done — Vue 3 dashboard
9	Date-based filter to view historical results	Done
10	Allow users to select a date range	Done
11	Display results based on selected dates	Done
12	Enable download of results (CSV/Report)	Done
13	Total unit test cases	Done
14	Unit test pass %	Done
15	Avg unit test execution time	Done
16	Docker support	Done